

Incident Memory: A Self-Improving Memory for On-Call Teams

Semantic search, LLM reasoning, and long-term learning that turn every resolved incident into a smarter response to the next one.

The on-call knowledge problem

Every engineering organization repeats itself. The same class of incident - a connection pool exhausted under load, a certificate silently expiring, a worker leaking memory - surfaces again months later, and the knowledge of how it was fixed is scattered across post-mortems, chat threads, and the memories of people who may not even be on the rotation anymore. On-call engineers lose hours rediscovering what the team already knew. This project, Incident Memory, is a working demonstration of a better pattern: an AI system that remembers. It combines semantic retrieval, large language models, and a persistent learning layer so that every resolved incident makes the next response faster and more precise.

What the system does

Incident Memory is a compact Flask application, backed by Postgres and Qdrant, that wraps the complete memory-compounds loop in one interface. An engineer describes a new incident; the system embeds that description, finds the most similar incidents it has ever seen, asks an LLM to propose a fix grounded in those historical matches, and lets the engineer approve or edit the suggestion. When the fix is stored, it becomes part of the memory - the very next similar incident will retrieve it. Memory literally compounds: each resolution adds one vector and one structured record that makes future suggestions smarter.

Architecture

The system is deliberately simple, with four cooperating pieces:

- Postgres is the source of truth. Every incident is a row with title, description, root cause, resolution, service, severity, status, and timestamp.

- Qdrant is the vector index. Each incident is embedded and stored as a point keyed by the same id as its Postgres row, searched by cosine similarity.

- An LLM (Gemini by default, OpenAI as an alternative) generates suggestions. It is prompted with the current incident, the ranked past matches, and, when available, recalled long-term memories.

- Hindsight is a persistent long-term memory bank that retains, recalls, and reflects on resolved incidents, providing a second, more durable layer of learning.

All configuration flows from a single .env file, and every module reuses the same database, embedding, and vector client setup - there is no duplicated connection code. The embedding step uses Gemini's free tier by default and falls back to OpenAI transparently, with retry logic and clear troubleshooting messages when a key is invalid or a model name has been retired.

How a query flows through the system

The pipeline is easy to follow. When an engineer submits a description, the app embeds it with the same model used at ingest time, then searches Qdrant for the top similar past incidents above a similarity threshold. For each hit, it pulls the full record back out of Postgres. Next, a prompt is assembled that contains the current incident, the ranked historical matches with their root causes and resolutions, and any experiences recalled from Hindsight. The LLM responds with a structured suggestion: probable root cause, investigation steps, recommended resolution, the historical evidence it matched on, confidence, and a rollback plan. The engineer edits or approves that text and marks the incident resolved. A single ordered write inserts the new row in Postgres and upserts the vector in Qdrant; from that moment the incident is searchable and citable as evidence for future queries.

The long-term learning layer

Beyond the vector index, the project keeps a persistent learning layer in Hindsight, a memory system that turns raw content into structured, searchable facts. Resolved incidents are retained as experiences - their service, severity, root cause, resolution, and any lesson - together with structured metadata, and deduplicated so re-resolving an incident never pollutes the bank. Every new query recalls relevant memories, and when several match, Hindsight reflects over them to synthesize the lessons they point to; those insights are fed back into the LLM prompt, so recommendations improve with every incident the team closes. The layer is guarded and optional: if the service or configuration is unavailable, the core search-and-suggest loop keeps working exactly as before.

The operator experience

A single-page Agent Brain HUD makes the machinery visible. The interface is a dark, glassmorphic dashboard with three panels: an incident intake form, the AI fix and resolution workspace, and the memory bank. A live pipeline trace animates through embedding, recall, reranking, and LLM synthesis with per-step timings, so an engineer can see exactly where the response comes from. Search results carry real similarity scores, and the memory bank shows every incident with severity badges, filters, and one-click delete and restore actions. A command palette, export button, telemetry log, and a toggle for the Hindsight layer round out a UI that feels less like a prototype and more like a control room.

Durability and lifecycle management

The project treats memory as a first-class asset. A backup module exports a complete snapshot - every Postgres row, its stored vector, and the embedding metadata - into a single JSON file, and can restore it with original ids and vectors intact, re-embedding only when the model has changed. Import is an upsert, never a wipe, and the database id sequence is re-synced so new inserts never collide. Incidents can be soft-deleted (instantly removed from search, fully undoable), restored, or permanently purged, and the ordering of writes is designed so the two stores can never silently desync.

Verification

Several test suites exercise the real API. An end-to-end walkthrough runs two complete memory-compounding cycles - resolving a payments incident, then confirming that a related checkout query retrieves and cites it - proving the learning loop in practice. A lifecycle test covers export, import, idempotency, delete, restore, purge,

and sequence safety, and a dedicated unit suite verifies the Hindsight wiring against a mocked SDK. The seed data itself is instructive: twelve realistic incidents across payments, checkout, auth, database, and caching services, deliberately written so that later incidents mirror earlier ones.

Limitations and next steps

The most valuable next work is capturing richer outcome metadata and adding integration tests against a real memory bank. Retained memories are already deduplicated, so repeated resolutions never create duplicates. But even today, the demo proves the core thesis: an incident response system that remembers its own history can answer every new page with evidence, confidence, and a faster path to resolution.

Conclusion

Incident Memory demonstrates a practical way to stop repeating ourselves. By combining semantic search with LLM reasoning and persistent learning, it turns the post-mortem from an end-of-incident chore into the beginning of the next fix. The pattern is small, observable, and easy to extend - and it is the kind of system that gets better the longer you use it.